

Judy.ai

Technical Documentation

Knowledge Base Platform

Powered by Amazon Bedrock & Strands Agents

Version 1.0

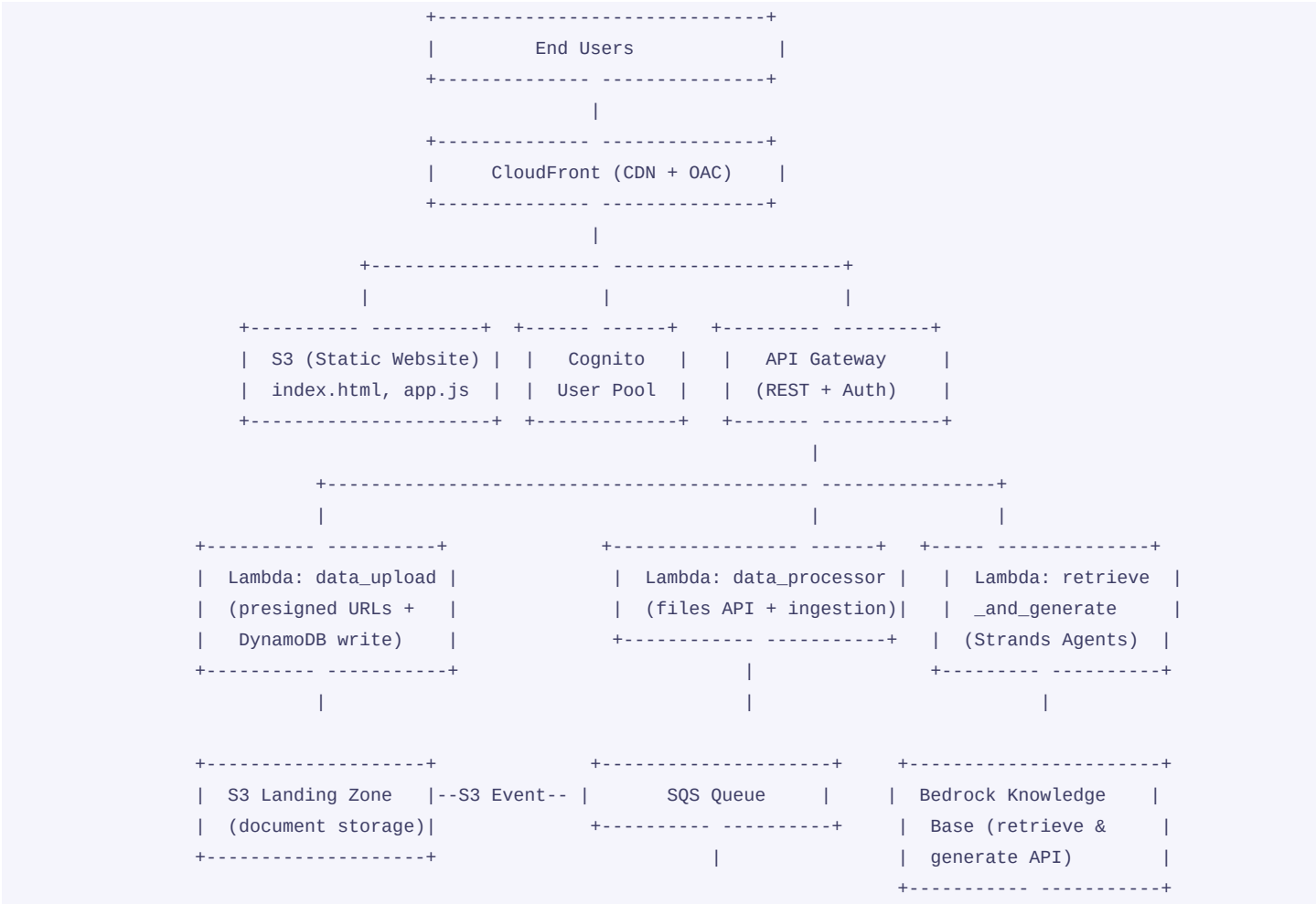
Confidential - Internal Use Only

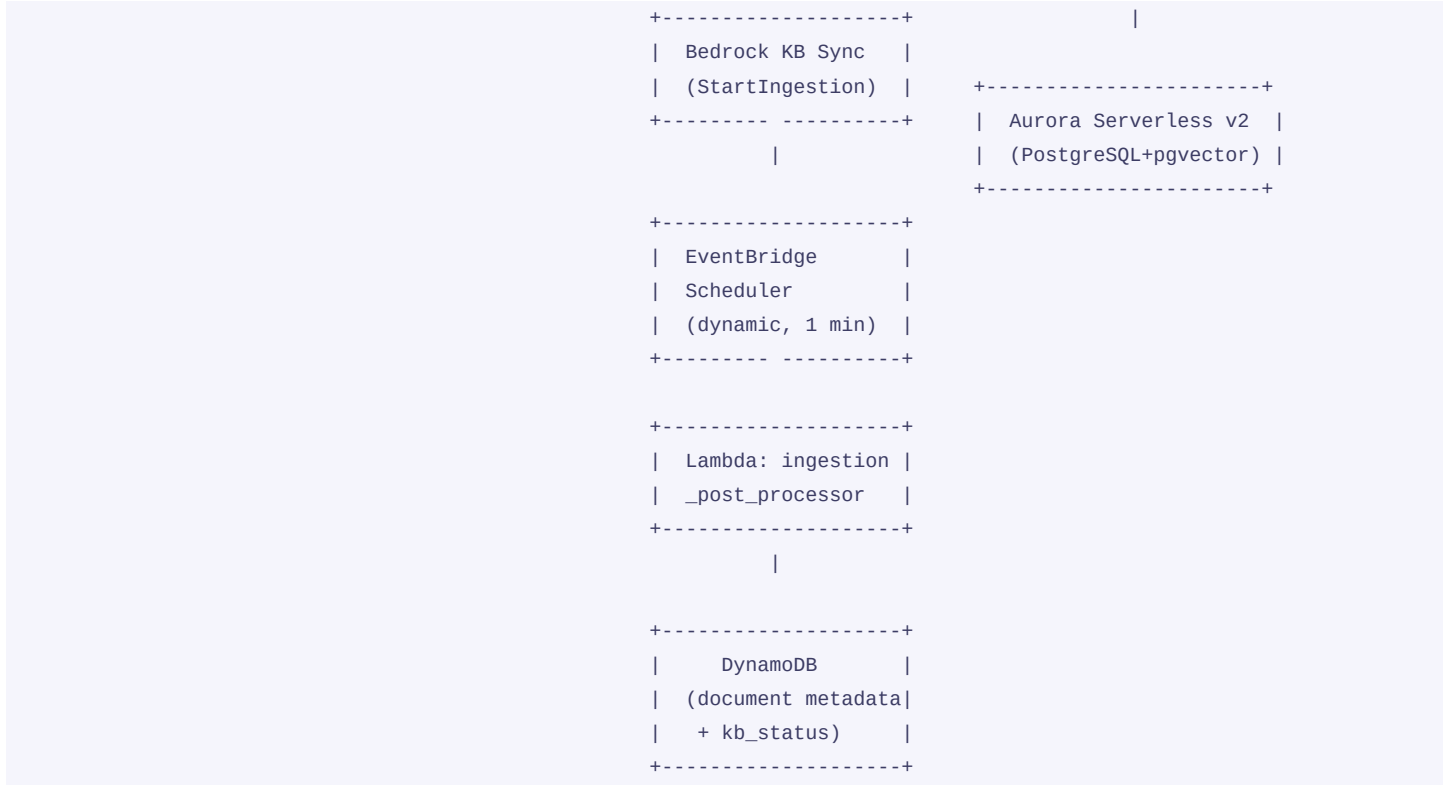
Judy.ai Knowledge Base - Technical Documentation

Table of Contents

- 1. Solution Architecture
- 2. Component Descriptions
- 3. Agentic RAG Pipeline
- 4. Data Flows
- 5. Configuration
- 6. Architectural Decisions
- 7. Terraform & Source Code Structure
- 8. DevOps: Deployment & Maintenance
- 9. Post-Deployment: Knowledge Base Setup with Default Data

Solution Architecture





Component Descriptions

Frontend

API Layer

Compute (Lambda Functions)

Data Storage

AI/ML

Event-Driven Components

Agentic RAG Pipeline

The `retrieve_and_generate` Lambda implements a 3-node agent graph using the Strands Agents SDK:

```
User Query
|

+-----+
| Intent Detector (Nova Lite) |
| - Classifies SAFE/UNSAFE   |
| - Blocks harmful queries   |
+-----+
      | (only if SAFE)

+-----+
| Retrieval Agent (Nova Lite) |
| - Calls retrieve_from_kb tool |
| - If no result: rephrases     |
| - If still no result: uses    |
|   general LLM knowledge       |
| - Checks citations for relevance|
+-----+
      |

+-----+
| Response Generator (Nova Pro) |
| - Formats tables, lists, bold |
| - Source attribution rules     |
| - Never fabricates information |
+-----+
      |

Final Response
```

Key Design Choices

- Fail-open safety: If intent detector output is ambiguous, query proceeds (only explicit "UNSAFE:" blocks)
- Citations-based detection: Uses Bedrock citations[] field (not heuristic text matching) to determine if KB had relevant results
- Module-level initialization: Graph is built once at Lambda cold start, reused across warm invocations
- Adaptive retry: boto3 clients configured with mode: adaptive for automatic throttle handling

Data Flows

Upload Flow

1. User drops file -> Frontend validates size (15MB max)
2. POST /upload -> data_upload Lambda

3. Lambda generates presigned URL + writes DynamoDB record (kb_status="loading")
4. Frontend PUTs file to S3 via presigned URL
5. S3 ObjectCreated event -> SQS -> data_processor Lambda
6. Lambda updates file_size_kb in DynamoDB
7. Lambda calls StartIngestionJob (Bedrock KB sync)
8. Lambda creates EventBridge Scheduler schedule (rate: 1 min)
9. Scheduler triggers ingestion_post_processor
10. Post-processor checks job status via GetIngestionJob
11. On COMPLETE: updates kb_status to "ingested", deletes schedule
12. On FAILED: updates kb_status to "failed", deletes schedule

Deletion Flow

1. User right-clicks file -> confirms deletion
2. DELETE /files/{id} -> data_processor Lambda
3. Lambda deletes file from S3
4. Lambda sets kb_status="deleting" in DynamoDB (record kept)
5. Lambda calls StartIngestionJob (re-sync)
6. Lambda creates EventBridge Scheduler schedule
7. Post-processor monitors job -> on COMPLETE: deletes DynamoDB record + schedule

Conflict Handling (Orphan Pattern)

- When StartIngestionJob throws ConflictException (job already running):
- Document left as "orphan" (no ingestion_job_id)
 - When the running job's post-processor completes:
 1. Processes its own tagged documents
 2. Checks for orphans (loading/deleting without job_id)
 3. If found: starts follow-up ingestion -> tags orphans -> creates new schedule
 4. Chain continues until all documents reach terminal state

Configuration

Terraform Variables

Resource Tagging

All resources are tagged with:

```
aws-apn-id = "pc:an2wsvdun8wdc006lw2pr8ag"
```

Applied globally via provider.aws.default_tags.

Aurora Configuration

Architectural Decisions

1. S3 -> SQS -> Lambda (not S3 -> Lambda direct)

Decision: Route S3 events through SQS before Lambda.

Rationale: S3 direct Lambda triggers don't support fan-out or retry with DLQ. SQS provides message durability, retry (10 attempts), and dead-letter queue for failed processing.

2. Dynamic EventBridge Scheduler (not static scheduled rule)

Decision: Lambda creates/deletes EventBridge Scheduler schedules dynamically per ingestion job.

Rationale: A static rule (running every N minutes perpetually) wastes compute when no jobs are active. Dynamic schedules exist only during active ingestion, then self-delete. Cost: \$0 when idle.

3. DynamoDB for document metadata (not S3 metadata)

Decision: Store file metadata in DynamoDB, not S3 object metadata or tags.

Rationale: DynamoDB supports fast scans, attribute updates (kb_status lifecycle), and doesn't require listing S3 objects (which is eventually consistent and slow).

4. Strands Agents graph pattern (not monolithic prompt)

Decision: Three-agent pipeline with separate models per task.

Rationale: Separation of concerns (safety, retrieval, formatting), different model sizes for different tasks (Lite for classification, Pro for generation), and fail-isolation (one agent failing doesn't crash the whole pipeline).

5. Bedrock Data Automation (BDA) parsing strategy

Decision: Use BDA for document parsing instead of the default parser.

Rationale: BDA provides superior OCR, table extraction, and multi-modal understanding for PDFs with images, charts, and complex layouts.

6. Cognito Implicit flow (not Authorization Code)

Decision: Use OAuth2 implicit flow for the static site.

Rationale: Simplest implementation for a pure client-side SPA with no backend token exchange. The token is stored in sessionStorage (cleared on tab close/logout). Acceptable for the current security requirements.

7. Job-document association (ingestion_job_id)

Decision: Tag each DynamoDB document with the specific ingestion_job_id that processes it.

Rationale: Without this, a global scan for "loading" documents could incorrectly mark files processed by different concurrent jobs. The association ensures per-job precision.

Terraform & Source Code Structure

Directory Layout

```
.
-- docs/
|   -- USER_MANUAL.md
|   +-- TECHNICAL_DOCUMENTATION.md
-- terraform/
|   -- main.tf           # Root module - wires all sub-modules
|   -- variables.tf      # Input variable declarations
|   -- outputs.tf        # Terraform outputs (URLs, IDs)
|   -- providers.tf      # AWS provider + default tags
|   -- terraform.tfvars  # Actual deployment values (gitignored)
|   -- terraform.tfvars.example # Template for new deployments
+-- modules/
|   -- s3/               # Website bucket + Landing Zone bucket
|   |   -- main.tf      # Buckets, CORS, website file uploads, EventBridge notification
|   |   -- variables.tf
|   |   +-- outputs.tf
|   -- cloudfront/      # Distribution + OAC + S3 bucket policy
|   |   -- main.tf
|   |   -- variables.tf
|   |   +-- outputs.tf
|   -- cognito/         # User Pool + Client + UI customization
|   |   -- main.tf
|   |   -- cognito-custom.css # Dark theme for hosted login UI
|   |   -- variables.tf
|   |   +-- outputs.tf
|   -- api_gateway/     # REST API + 4 routes + Cognito authorizer
|   |   -- main.tf
|   |   -- variables.tf
|   |   +-- outputs.tf
|   -- lambda/          # 4 Lambda functions + IAM + EventBridge Scheduler role
|   |   -- main.tf
|   |   -- variables.tf
|   |   +-- outputs.tf
|   -- aurora/          # Serverless v2 cluster + Secrets Manager
|   |   -- main.tf
|   |   -- variables.tf
|   |   +-- outputs.tf
|   -- bedrock_kb/      # Knowledge Base + S3 data source + BDA IAM
|   |   -- main.tf
```

```

|         | -- variables.tf
|         | +-- outputs.tf
|         | -- dynamodb/          # Documents metadata table
|         | -- main.tf
|         | -- variables.tf
|         | +-- outputs.tf
|         | -- sqs/            # Ingestion queue + DLQ + S3 notification + alarm
|         | -- main.tf
|         | -- variables.tf
|         | +-- outputs.tf
| +-- ssm/          # KB ID + DS ID parameters
|     -- main.tf
|     -- variables.tf
|     +-- outputs.tf
-- lambdas/
| -- data_upload/
| | +-- handler.py          # Presigned URL + DynamoDB metadata write
| -- data_processor/
| | +-- handler.py          # SQS handler + Files API + ingestion orchestration
| -- retrieve_and_generate/
| | +-- handler.py          # Strands Agents graph (3-node agentic RAG)
| +-- ingestion_post_processor/
|     +-- handler.py        # Job status polling + orphan follow-up
-- website/
| -- index.html            # Main SPA page (sidebar + 3 sections)
| -- callback.html         # Cognito OAuth callback (token capture)
| -- styles.css            # Full dark theme CSS (Judy.ai design system)
| -- app.js               # Application logic (upload, chat, files, sessions)
| -- auth.js              # Cognito auth (login redirect, logout, token management)
| -- logo-color.svg        # Judy.ai color logo (inline SVG in HTML)
| +-- config.js            # Auto-generated by Terraform (API URLs, Cognito config)
+-- sql/
    +-- setup_vector_db.sql # Aurora pgvector schema setup script

```

Key Terraform Resources by Module

DevOps: Deployment & Maintenance

Prerequisites

- Terraform >= 1.5.0
- AWS CLI v2 configured with credentials
- Access to Amazon Bedrock models (enable in the Bedrock console):
 - Cohere Embed Multilingual v3
 - Amazon Nova Pro v1
 - Amazon Nova Lite v1
- PostgreSQL client (psql) for database setup

Step 1: Configure Variables

```
cd terraform
cp terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars with your values (passwords, region, etc.)
```

Step 2: Deploy Infrastructure

```
terraform init
terraform plan    # Review changes
terraform apply   # Deploy (type 'yes' to confirm)
```

Expected output includes:

- cloudfront_distribution_url - Application URL
- cognito_user_pool_id - For user management
- knowledge_base_id - KB identifier
- aurora_cluster_endpoint - Database endpoint

Step 3: Setup Aurora Database

Connect to Aurora and run the SQL setup script:

```
# Get the cluster endpoint
ENDPOINT=$(terraform output -raw aurora_cluster_endpoint)

# Connect using psql
psql -h $ENDPOINT -U <aurora_master_username> -d ragdb

# Run the setup script (replace placeholders first!)
\i ../sql/setup_vector_db.sql
```

Critical: Replace BEDROCK_USERNAME and BEDROCK_PASSWORD in the SQL script with the exact values from your terraform.tfvars (aurora_bedrock_user_username and aurora_bedrock_user_password).

Step 4: Create Initial User

```
USER_POOL_ID=$(terraform output -raw cognito_user_pool_id)

aws cognito-idp admin-create-user \
  --user-pool-id $USER_POOL_ID \
  --username admin@yourcompany.com \
  --user-attributes Name=email,Value=admin@yourcompany.com Name=email_verified,Value=true \
  --temporary-password TempPassword123!
```

Step 5: Invalidate CloudFront Cache (after updates)

```
DIST_ID=$(terraform output -raw cloudfront_distribution_id)
```

```
aws cloudfront create-invalidation --distribution-id $DIST_ID --paths "/*"
```

Updating the Application

```
cd terraform
terraform apply # Detects file changes via etag/hash, re-uploads
# Then invalidate CloudFront cache for immediate effect
```

Destroying the Environment

```
cd terraform
terraform destroy # Type 'yes' to confirm
```

Warning: This deletes ALL resources including the Aurora database and S3 data. Ensure you have backups of any important data.

Post-Deployment: Knowledge Base Setup with Default Data

After infrastructure deployment, the Knowledge Base is empty. This section describes how to load default/seed data so users have content available immediately.

Overview

The process:

1. Prepare default documents (PDF/DOCX)
2. Upload them to the S3 Landing Zone
3. Trigger a Knowledge Base synchronization job
4. Validate that data is queryable via the Retrieve & Generate API

Step 1: Prepare Default Documents

Gather the PDF/DOCX files that should be pre-loaded. Place them in a local directory:

```
mkdir -p default-data/
# Copy your default documents here
cp /path/to/document1.pdf default-data/
cp /path/to/document2.pdf default-data/
```

Step 2: Upload to S3 Landing Zone

Upload documents to the landing zone bucket using the same path pattern the application uses (uploads/{uuid}/{filename}):

```
# Get the bucket name
BUCKET=$(terraform output -raw landing_zone_bucket)
```

```
# Upload each file with a unique UUID prefix
for file in default-data/*; do
  UUID=$(uuidgen | tr '[:upper:]' '[:lower:]')
  FILENAME=$(basename "$file")
  aws s3 cp "$file" "s3://${BUCKET}/uploads/${UUID}/${FILENAME}"
  echo "Uploaded: uploads/${UUID}/${FILENAME}"
done
```

Note: Uploading via S3 directly (not the app UI) means DynamoDB records won't be created for these files. They will exist in the KB but not appear in the "Files" page. If you want them visible in the UI, upload via the application's upload page instead.

Step 3: Trigger Knowledge Base Synchronization

```
# Get KB and DS IDs
KB_ID=$(terraform output -raw knowledge_base_id)
DS_ID=$(terraform output -raw data_source_id)

# Start ingestion job
aws bedrock-agent start-ingestion-job \
  --knowledge-base-id $KB_ID \
  --data-source-id $DS_ID \
  --region us-east-1
```

Note the ingestionJobId from the response.

Step 4: Monitor Synchronization Progress

```
# Check job status (repeat until status is "COMPLETE")
aws bedrock-agent get-ingestion-job \
  --knowledge-base-id $KB_ID \
  --data-source-id $DS_ID \
  --ingestion-job-id <JOB_ID> \
  --region us-east-1
```

Expected output when complete:

```
{
  "ingestionJob": {
    "status": "COMPLETE",
    "statistics": {
      "numberOfDocumentsScanned": 5,
      "numberOfNewDocumentsIndexed": 5,
      "numberOfDocumentsFailed": 0
    }
  }
}
```

If numberOfDocumentsFailed > 0: Check the failureReasons field for details. Common issues:

- File too large (max 50MB for Bedrock KB)
- Unsupported format

- IAM permission issues

Step 5: Validate with Retrieve & Generate API

Test that the default data is queryable:

```
# Test a query against the knowledge base
aws bedrock-agent-runtime retrieve-and-generate \
  --input '{"text": "What are the main topics covered in the documents"}' \
  --retrieve-and-generate-configuration '{
    "type": "KNOWLEDGE_BASE",
    "knowledgeBaseConfiguration": {
      "knowledgeBaseId": "'$KB_ID'",
      "modelArn": "arn:aws:bedrock:us-east-1::foundation-model/amazon.nova-pro-v1:0"
    }
  }' \
  --region us-east-1
```

Verify:

- The response contains relevant information from your uploaded documents
- Citations reference the uploaded S3 keys
- No errors in the response

Step 6: Validate via the Application UI

1. Open the application URL in a browser
2. Log in with the admin user
3. Navigate to Chat
4. Ask a question about the default documents
5. Verify you receive a relevant, sourced answer

Alternative: Upload Default Data via the Application UI

If you want the files to appear in the Files page with proper metadata:

1. Log into the application
2. Navigate to Upload
3. Upload each default document through the drag-and-drop interface
4. Navigate to Files and wait for status to change to "Ingested"
5. Test via Chat

This approach is simpler but requires manual effort per file.

Troubleshooting Default Data Setup